

Arquitectura de Comunicación Frontend-Backend

Documento de análisis y propuesta para la separación del frontend y backend en el proyecto Gestor-Horarios.

Contexto

El equipo

Rol	Persona	¿Necesita Docker?
Líder Backend (Tech-Lead)	Luis Rojas	✓ Sí (con WSL2)
Líder Frontend (Tech-Lead)	Daniel Reyna	✗ Sin WSL2
Desarrolladora Frontend	Paola Peña	? Opcional
Desarrolladora Backend	Nicole Sereno	✓ Sí (con WSL2)
Middleware / QA	Manuel Garcia	? Opcional

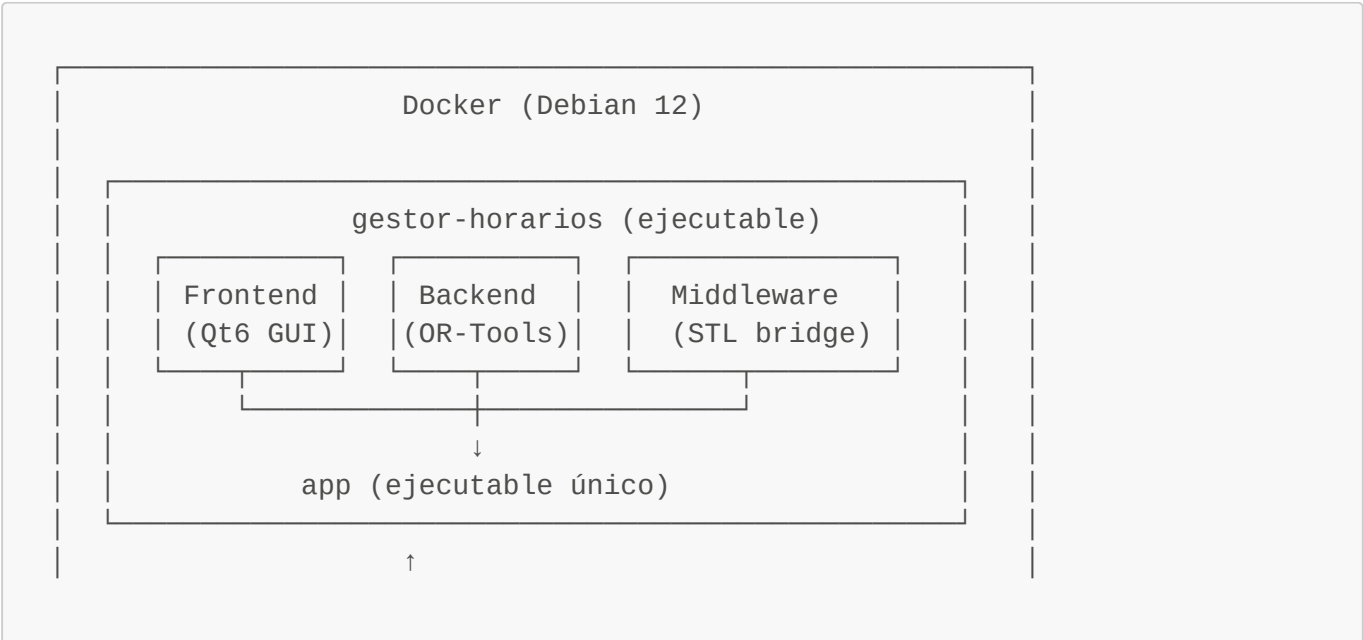
El problema

Daniel tiene una versión modificada de Windows **sin acceso a la capa de compatibilidad WSL2**. Esto significa que no puede ejecutar el contenedor Docker actual (`debian:12-slim` con OR-Tools + Qt6), que es el entorno de desarrollo unificado del proyecto.

El objetivo

Generar un programa funcional para PCs Windows, permitiendo que **todo el equipo pueda desarrollar simultáneamente** sin depender de un entorno único.

La arquitectura actual



X11 forwarding para GUI

Problema: Si no tienes WSL2, no puedes ni siquiera iniciar el proyecto.

Las opciones

Opción 0: Status Quo — Monolito en Docker

Principio

Todo el código se compila en un solo binario dentro del contenedor Docker. El frontend y backend se comunican mediante llamadas a funciones directas en memoria.

Cómo se comunican frontend y backend

Actualmente es un **monolito**: frontend, backend y middleware son librerías estáticas que se linkan juntas en un solo ejecutable. La comunicación es directa en memoria:

```
// — backend/schedule_model.hpp —
#pragma once
#include <vector>
#include "types.hpp"

namespace backend {

class ScheduleModel {
public:
    [[nodiscard]] auto optimizar(DatosEntrada const& datos) ->
ResultadoHorario;
};

} // namespace backend

// — middleware/schedule_service.hpp —
#pragma once
#include "backend/schedule_model.hpp"

namespace middleware {

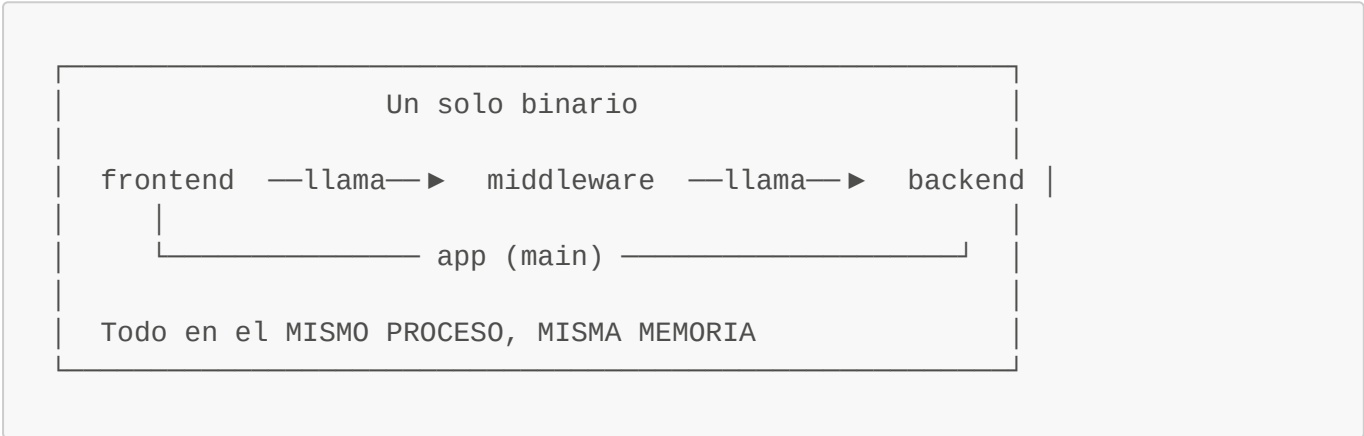
class ScheduleService {
    backend::ScheduleModel modelo_;
public:
    auto generarHorario(DatosEntrada const& datos) -> ResultadoHorario {
        return modelo_.optimizar(datos);
    }
};

} // namespace middleware
```

```
// — frontend/main_window.cpp —
#include "middleware/schedule_service.hpp"

void MainWindow::onGenerarClick() {
    auto datos = formulario_.obtenerDatos();
    middleware::ScheduleService servicio;
    auto resultado = servicio.generarHorario(datos);
    mostrarResultado(resultado);
}
```

Diagrama



Pros

- **Simplicidad máxima:** un solo binario, un solo proceso
- **Performance óptima:** llamadas directas a funciones, cero overhead de serialización
- **Sin nuevas dependencias:** no hay que agregar librerías de red/IPC
- **Debugging simple:** un solo depurador, un solo stack trace

Contras

- **Daniel no puede desarrollar:** necesita Docker con WSL2 para compilar y ejecutar
- **X11 forwarding es lento:** la GUI corre a través de la red, con latencia visible
- **Un crash lo tira todo:** un error en backend derriba la interfaz de usuario
- **Acoplamiento total:** frontend y backend no pueden evolucionar independientemente
- **QA depende de Docker:** Manuel necesita el contenedor para cualquier prueba

Flujo de trabajo

Persona	¿Puede trabajar?	¿Cómo?
Luis	✓	Docker con WSL2, X11 forwarding
Daniel	✗	Bloqueado — sin WSL2 no puede
Paola	✗	Bloqueada si no tiene WSL2

Persona	¿Puede trabajar?	¿Cómo?
Nicole	✓	Docker con WSL2
Manuel	?	Solo si tiene WSL2

Opción A: Qt Local Sockets (IPC Nativo) — RECOMENDADA

Principio

Frontend y backend son procesos independientes que se comunican a través de **named pipes** (IPC nativo del sistema operativo) usando `QLocalServer` y `QLocalSocket` de Qt. No hay TCP, no hay HTTP, no hay web servers. Es comunicación entre procesos pura.

Cómo se comunican frontend y backend

El backend se convierte en un **servicio de consola** (sin GUI) que expone un servidor de named pipes. El frontend se conecta a ese pipe y envía/recibe mensajes JSON.

```
// — backend/main.cpp (servicio) —
#include <QCoreApplication>
#include <QLocalServer>
#include <QJsonDocument>
#include <QJsonObject>
#include "schedule_model.hpp"

int main(int argc, char *argv[]) {
    QCoreApplication app(argc, argv);

    backend::ScheduleModel modelo;
    QLocalServer server;

    // El backend escucha en un pipe con nombre
    server.listen("gestor-horarios-backend");

    QObject::connect(&server, &QLocalServer::newConnection, [&] {
        auto* socket = server.nextPendingConnection();

        // Cuando llegan datos del frontend
        QObject::connect(socket, &QLocalSocket::readyRead, [socket,
&modelo] {
            QByteArray datos = socket->readAll();
            QJsonDocument doc = QJsonDocument::fromJson(datos);

            // Procesar con OR-Tools
            auto entrada = jsonADatosEntrada(doc.object());
            auto resultado = modelo.optimizar(entrada);

            // Responder con JSON
            QJsonDocument respuesta(datosEntradaAJson(resultado));
            socket->write(respuesta.toJson());
        });
    });
}
```

```

    });
});

return app.exec();
}

```

```

// — frontend/backend_client.hpp —
#pragma once
#include <QLocalSocket>
#include <QJsonDocument>
#include <QJsonObject>

class BackendClient {
    QLocalSocket socket_;
public:
    BackendClient() {
        socket_.connectToServer("gestor-horarios-backend");
    }

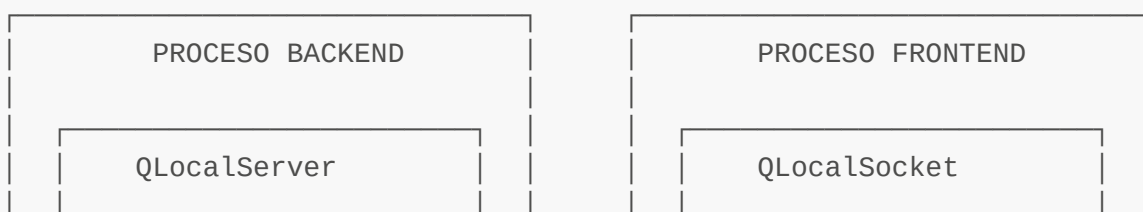
    auto enviarPeticion(QJsonObject const& datos) -> QJsonObject {
        QJsonDocument doc(datos);
        socket_.write(doc.toJson());
        socket_.waitForBytesWritten();
        socket_.waitForReadyRead();
        return QJsonDocument::fromJson(socket_.readAll()).object();
    }
};

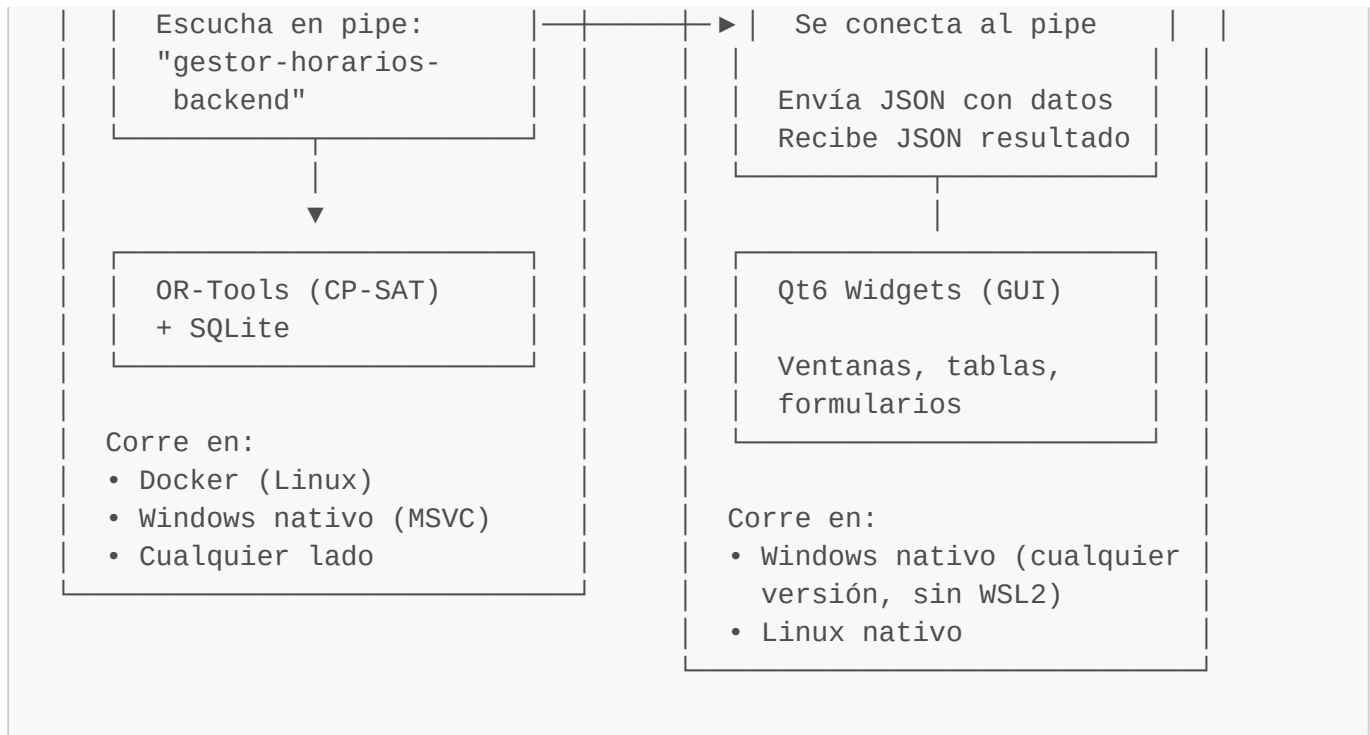
// — frontend/main_window.cpp —
#include "backend_client.hpp"

void MainWindow::onGenerarClick() {
    BackendClient cliente;
    auto respuesta = cliente.enviarPeticion({
        {"docentes", formulario_.docentesAJson()},
        {"aulas",    formulario_.aulasAJson()},
        {"secciones", formulario_.seccionesAJson()}
    });
    mostrarResultado(respuesta);
}

```

Diagrama





Middleware en esta arquitectura

El middleware se divide en tres partes:

```

middleware/
├── common/           ← Tipos compartidos (estructuras, serialización JSON)
│   ├── types.hpp     → DatosEntrada, ResultadoHorario, Docente, Aula...
│   └── serialization.hpp → Conversión a/desde QJsonObject
├── server/           ← Lado del backend
│   ├── api_handler.hpp → Recibe JSON, llama al backend, devuelve JSON
│   └── server_main.cpp → Punto de entrada del servicio
└── client/           ← Lado del frontend
    ├── backend_client.hpp → QLocalSocket wrapper
    └── request_builder.hpp → Construye peticiones tipadas
  
```

Pros

- **Cero dependencias nuevas:** Qt6::Core ya está en el proyecto — `QLocalServer` y `QLocalSocket` vienen incluidos
- **Cross-platform real:** Windows Named Pipes en Windows, Unix Domain Sockets en Linux
- **Sin HTTP:** IPC nativo del SO, sin overhead de protocolos web
- **Rápido:** la comunicación es a nivel kernel, no pasa por pila TCP/IP
- **Daniel puede desarrollar frontend:** corre Qt6 nativo en Windows, backend en otro proceso
- **QA puede testear el backend:** Manuel escribe un script que se conecta al pipe
- **Aislamiento:** si el backend crashea, el frontend no se cae (reconecta)
- **Rendición de cuentas clara:** cada proceso tiene su ciclo de vida

Contras

- **Dos procesos que gestionar:** hay que arrancar el backend antes que el frontend
- **Serialización JSON:** overhead mínimo comparado con llamadas directas, pero existe
- **No funciona sobre red:** los named pipes son locales a la máquina (aunque para este proyecto es perfecto)
- **El backend necesita Qt6::Core:** actualmente el backend solo linkea OR-Tools, habría que agregar Qt6::Core (que ya está instalado en el contenedor)

Flujo de trabajo

Persona	¿Puede trabajar?	¿Cómo?
Luis	✓	Backend en Docker (WSL2) + Frontend nativo
Daniel	✓	Frontend nativo en Windows. Backend en otro proceso (mock o compilado)
Paola	✓	Frontend nativo en Windows. Backend mockeado para desarrollo visual
Nicole	✓	Backend en Docker (WSL2). Testea con script contra el pipe
Manuel (QA)	✓	Sin Docker. Escribe scripts que hablan con el backend por el pipe. Testea frontend nativo

QA workflow con Opción A:

1. `git pull develop`
2. Para testear backend:
 - a. Arranca backend (docker-up o binario nativo)
 - b. Ejecuta suite de tests: `./test_backend` (hablan por el pipe)
 - c. O script Python que se conecta al named pipe
3. Para testear frontend:
 - a. Compila frontend nativo (Qt6 en Windows)
 - b. Ejecuta frontend → se conecta al backend automáticamente
4. Integration test: frontend + backend juntos

Opción B: TCP Local (QTcpServer / QTcpSocket)

Principio

Frontend y backend se comunican a través de **TCP loopback** (localhost). Es la misma idea que la Opción A, pero usando TCP en vez de named pipes.

Cómo se comunican frontend y backend

```
// — backend/main.cpp —
#include <QCoreApplication>
#include <QTcpServer>
#include <QTcpSocket>
#include <QJsonDocument>

int main(int argc, char *argv[]) {
    QCoreApplication app(argc, argv);
    QTcpServer server;

    server.listen(QHostAddress::LocalHost, 8080);
    //                               ^^^^
    // Puerto TCP en localhost

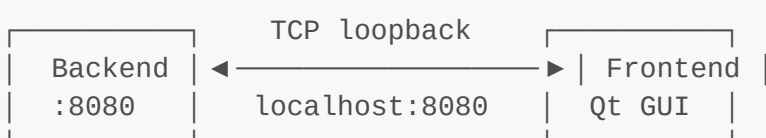
    QObject::connect(&server, &QTcpServer::newConnection, [&] {
        auto* socket = server.nextPendingConnection();
        QObject::connect(socket, &QTcpSocket::readyRead, [socket] {
            QByteArray data = socket->readAll();
            // Procesar y responder...
            socket->write(respuestaJson);
        });
    });

    return app.exec();
}
```

```
// — frontend/backend_client.hpp —
#include <QTcpSocket>

class BackendClient {
    QTcpSocket socket_;
public:
    BackendClient() {
        socket_.connectToHost(QHostAddress::LocalHost, 8080);
    }
    // ... mismo patrón que QLocalSocket
};
```

Diagrama



Pros

- **Cero dependencias nuevas** (Qt6::Network)
- **Puedes testear con curl**: `curl localhost:8080 -d '{"datos":...}'`
- **Portabilidad futura**: si algún día el backend no está en localhost, cambiar la IP es trivial
- **Idéntico en Windows y Linux**: TCP funciona igual en ambos

Contras

- **Overhead TCP**: aunque es loopback, sigue pasando por la pila TCP/IP
- **Puertos que gestionar**: conflicto de puertos si otro programa usa el mismo
- **Seguridad**: cualquier proceso en localhost puede conectarse (no es un problema real para una app desktop)
- **Más complejo que named pipes**: necesita dirección y puerto vs un nombre simple

Flujo de trabajo

Persona	¿Puede trabajar?
Luis	✓ Igual que Opción A
Daniel	✓ Frontend nativo, backend en localhost:8080
Paola	✓
Nicole	✓
Manuel (QA)	✓ Puede testear con <code>curl</code> directamente

Opción C: HTTP REST (cpp-http lib / libcurl)

Principio

Frontend y backend se comunican mediante una API REST sobre HTTP. El backend expone endpoints, el frontend los consume como si fuera un cliente web.

Cómo se comunican frontend y backend

```
// — backend/server.cpp —
#include "http lib.h"

void iniciarServidor() {
    http lib::Server svr;

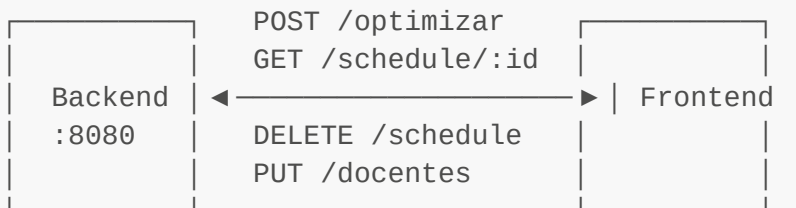
    svr.Post("/optimizar", [] (const http lib::Request& req,
http lib::Response& res) {
        auto datos = json::parse(req.body);
        auto resultado = modelo.optimizar(datos);
        res.set_content(resultado.dump(), "application/json");
    });
}
```

```
svr.listen("0.0.0.0", 8080);
}
```

```
// — frontend/api_client.cpp —
#include <curl/curl.h>

class ApiClient {
public:
    auto optimizar(DatosEntrada const& datos) -> ResultadoHorario {
        CURL* curl = curl_easy_init();
        curl_easy_setopt(curl, CURLOPT_URL,
            "http://localhost:8080/optimizar");
        // ... enviar JSON, recibir respuesta
    }
};
```

Diagrama



Pros

- **Estándar:** todo programador conoce REST/HTTP
- **Testing máximo:** `curl`, Postman, scripts — cualquier herramienta HTTP funciona
- **Documentación clara:** los endpoints son auto-documentados
- **Escalable:** si algún día necesitan un frontend web, ya tienen la API
- **Separación de concerns clara:** el contrato es la URL + JSON

Contras

- **Nueva dependencia:** hay que integrar `cpp-httpLib` (servidor) y `libcurl` (cliente)
- **Overhead HTTP:** headers, métodos, códigos de estado — más complejo de lo necesario para una app local
- **Configuración de servidor:** hay que gestionar puertos, CORS (innecesario en localhost), timeouts
- **Sienta a "web":** para una app de escritorio, sentir que estás corriendo un web server puede ser incómodo
- **Latencia:** cada petición HTTP implica handshake TCP + parsing HTTP

Flujo de trabajo

Persona	¿Puede trabajar?
Luis	✓
Daniel	✓
Paola	✓
Nicole	✓
Manuel (QA) ✓ Ideal para QA — puede testear cada endpoint con curl	

Opción D: Shared Library / Plugin (C ABI)

Principio

El backend se compila como una **librería dinámica** (.dll en Windows, .so en Linux). El frontend la carga en tiempo de ejecución mediante `QLibrary` o `dlopen()` y llama a las funciones directamente.

Cómo se comunican frontend y backend

```
// — backend/public_api.h — (interfaz C estable)
#pragma once

#ifdef _WIN32
    #define EXPORT __declspec(dllexport)
#else
    #define EXPORT __attribute__((visibility("default")))
#endif

extern "C" {

    struct Resultado { char const* json; int longitud; };
    struct Entrada { char const* json; int longitud; };

    EXPORT Resultado optimizar_horario(Entrada entrada);
    EXPORT void liberar_resultado(Resultado* res);

} // extern "C"
```

```
// — frontend/backend_loader.cpp —
#include <QLibrary>

class BackendLoader {
    QLibrary lib_;
    using OptimizarFn = Resultado (*)(Entrada);

public:
    bool cargar() {
        lib_.setFileName("backend.dll"); // o libbackend.so
```

```

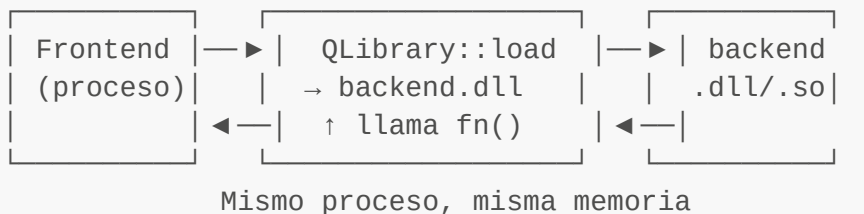
        return lib_.load();
    }

    auto optimizar(DatosEntrada const& datos) -> ResultadoHorario {
        auto fn = (OptimizarFn)lib_.resolve("optimizar_horario");
        if (!fn) throw std::runtime_error("No se pudo cargar
optimizar_horario");

        auto json = datosAJson(datos);
        Entrada e {json.c_str(), (int)json.size()};
        Resultado res = fn(e);
        // ... convertir resultado
    }
};

```

Diagrama



Pros

- **Máxima performance:** llamadas directas a funciones, cero serialización
- **Sin IPC:** todo en el mismo proceso, sin pipes ni sockets
- **Distribución simple:** el .exe del frontend + la .dll del backend

Contras

- **Daniel necesita la .dll:** no puede compilar el backend en su máquina. Depende de que CI o Luis le provean la .dll para Windows
- **Un crash lo mata todo:** si backend.dll explota, también se cae el frontend
- **ABI inestable:** cambiar el backend requiere recompilar la interfaz C y redistribuir la .dll
- **QA no puede testear el backend aislado:** necesita el frontend para probar
- **Versionado complejo:** qué pasa si el frontend espera v2 de la API pero la .dll es v1
- **No hay aislamiento:** backend y frontend compiten por memoria, handles, etc.

Flujo de trabajo

Persona	¿Puede trabajar?
Luis	✅ Puede compilar todo
Daniel	🟡 Solo frontend, necesita .dll de CI o de Luis

Persona	¿Puede trabajar?
Paola	🟡 Igual que Daniel
Nicole	✅ Compila todo en Docker
Manuel (QA)	❌ No puede testear el backend aislado

Opción E: gRPC

Principio

Frontend y backend se comunican mediante **gRPC**, un framework de RPC (Remote Procedure Call) con contratos definidos en **Protocol Buffers** (**.proto**). Los mensajes son binarios, eficientes y tipados.

Cómo se comunican frontend y backend

```
// — proto/gestor_horarios.proto —
syntax = "proto3";

service GestorHorarios {
    rpc Optimizar(OptimizarRequest) returns (OptimizarResponse);
}

message Docente {
    string nombre = 1;
    repeated string materias = 2;
    repeated int32 disponibilidad = 3;
}

message OptimizarRequest {
    repeated Docente docentes = 1;
    // ... más campos
}

message OptimizarResponse {
    string horario_json = 1;
    bool exito = 2;
    string error = 3;
}
```

```
// Código generado por protoc:
// - gestor_horarios.pb.h (mensajes)
// - gestor_horarios.grpc.pb.h (servicio/cliente)

// — backend/server.cpp —
class GestorHorariosImpl final : public GestorHorarios::Service {
    Status Optimizar(ServerContext*, const OptimizarRequest* req,
```

```

        OptimizarResponse* res) override {
    auto resultado = modelo_.optimizar(desdeProto(*req));
    res->set_horario_json(resultado.aJson());
    res->set_exito(true);
    return Status::OK;
}
};

```

```

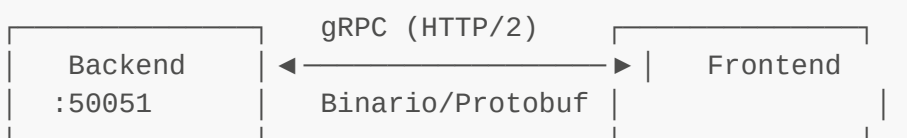
// — frontend/client.cpp —
auto stub = GestorHorarios::NewStub(grpc::CreateChannel(
    "localhost:50051", grpc::InsecureChannelCredentials()));

OptimizarRequest req;
req.add_docentes()->set_nombre("Prof. Pérez");
// ...

OptimizarResponse res;
stub->Optimizar(context, req, &res);

```

Diagrama



Pros

- **Contrato formal:** el archivo `.proto` es la fuente de verdad de la interfaz
- **Tipado fuerte:** los mensajes son generados, no hay errores de serialización
- **Streaming bidireccional:** soporta flujos de datos en tiempo real
- **Eficiente:** Protobuf es binario, más compacto que JSON
- **Multi-lenguaje:** si algún día hacen un frontend web o mobile, ya tienen el `.proto`

Contras

- **Dependencia pesada:** gRPC C++ no es trivial de integrar — requiere `protoc`, `grpc-cpp`, `abseil`
- **Build complejo:** hay que generar código C++ del `.proto` como parte de la compilación
- **En Windows es una odisea:** compilar gRPC para Windows (especialmente con Qt6 y OR-Tools) es significativamente complejo
- **Overkill para este proyecto:** gRPC brilla en microservicios distribuidos, no en una app desktop local
- **La imagen Docker crece:** gRPC añade cientos de MB a la imagen

Flujo de trabajo

Persona	¿Puede trabajar?
Luis	● Sí, pero más complejo de configurar
Daniel	● Compilar gRPC en Windows es un desafío
Paola	✗ Alta probabilidad de problemas de build
Nicole	● Más complejo en Docker también
Manuel (QA)	● Puede testear con grpcurl

Opción F: ZeroMQ

Principio

Frontend y backend se comunican mediante **ZeroMQ** (ØMQ), una librería de mensajería asíncrona que abstrae sockets tradicionales en patrones de mensajería (Request-Reply, Pub-Sub, Push-Pull, etc.).

Cómo se comunican frontend y backend

```
// — backend/server.cpp —
#include <zmq.hpp>

int main() {
    zmq::context_t ctx(1);
    zmq::socket_t socket(ctx, zmq::socket_type::rep);
    socket.bind("tcp://*:5555");

    while (true) {
        zmq::message_t request;
        socket.recv(request, zmq::recv_flags::none);

        auto datos = json::parse(request.to_string());
        auto resultado = modelo.optimizar(datos);

        zmq::message_t reply(resultado.dump());
        socket.send(reply, zmq::send_flags::none);
    }
}
```

```
// — frontend/client.cpp —
#include <zmq.hpp>

class BackendClient {
    zmq::context_t ctx_;
    zmq::socket_t socket_;
public:
    BackendClient() : socket_(ctx_, zmq::socket_type::req) {
```

```

        socket_.connect("tcp://localhost:5555");
    }

    auto optimizar(DatosEntrada const& datos) -> ResultadoHorario {
        zmq::message_t request(datos.aJson());
        socket_.send(request, zmq::send_flags::none);

        zmq::message_t reply;
        socket_.recv(reply, zmq::recv_flags::none);
        return ResultadoHorario::desdeJson(reply.to_string());
    }
};

```

Diagrama



Patrón: Request-Reply (bloqueante, síncrono)
 Alternativa: PUB-SUB (eventos), PUSH-PULL (tareas)

Pros

- **Ligero:** ZeroMQ es pequeño, sin dependencias externas
- **Multi-patrón:** REQ/REP, PUB/SUB, PUSH/PULL, PAIR — flexible
- **Rápido:** mensajería optimizada, bypass de overhead TCP
- **Multi-plataforma:** funciona en Windows, Linux, macOS
- **Idiomático:** ~100 líneas de código para un cliente-servidor completo

Contras

- **Nueva dependencia:** hay que integrar `libzmq` y `cppzmq` (header-only)
- **No hay contrato formal:** los mensajes son strings, no hay validación de tipos
- **Debugging:** curva de aprendizaje para entender los patrones de mensajería
- **QA necesita herramientas extra:** no es tan testable como HTTP con curl
- **Documentación escasa en C++:** la mayoría de ejemplos son en Python

Flujo de trabajo

Persona	¿Puede trabajar?
Luis	✓
Daniel	✓ (ZeroMQ tiene soporte Windows)
Paola	● Curva de aprendizaje

Persona	¿Puede trabajar?
Nicole	
Manuel (QA)	 Necesita scripts ZMQ, no puede usar curl

Opción G: Archivos Compartidos (File-based IPC)

Principio

Frontend y backend se comunican a través del **sistema de archivos**: el frontend escribe un JSON, el backend lo lee, procesa, y escribe la respuesta en otro archivo.

Cómo se comunican frontend y backend

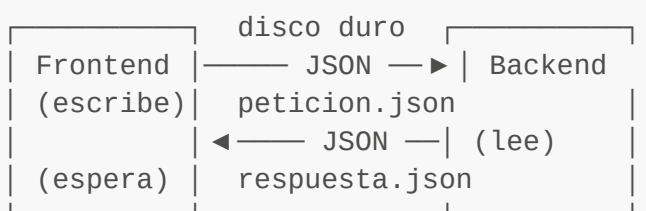
```
// — frontend —
void MainWindow::onGenerarClick() {
    QFile archivo("peticiones/pendiente_001.json");
    archivo.open(QIODevice::WriteOnly);
    archivo.write(QJsonDocument(datos).toJson());
    archivo.close();

    // Esperar a que el backend procese...
    QFile resultado("respuestas/resultado_001.json");
    // Polling hasta que exista...
}
```

```
// — backend —
int main() {
    while (true) {
        for (auto& archivo : QDir("peticiones/").entryList()) {
            QFile peticion("peticiones/" + archivo);
            peticion.open(QIODevice::ReadOnly);
            auto datos = QJsonDocument::fromJson(peticion.readAll());
            peticion.remove();

            auto resultado = modelo.optimizar(datos);
            QFile resp("respuestas/" + archivo);
            resp.write(QJsonDocument(resultado).toJson());
        }
        QThread::sleep(1); // Polling
    }
}
```

Diagrama



También posible con SQLite como buzón compartido.

Pros

- **Máxima simplicidad conceptual:** solo archivos, cero IPC
- **Persistencia gratuita:** las peticiones quedan registradas
- **Debugging trivial:** abres los archivos y ves qué se mandó

Contras

- **LENTO:** operaciones de disco, polling, sincronización
- **Condiciones de carrera:** qué pasa si frontend y backend escriben a la vez
- **Sin notificaciones:** el frontend tiene que hacer polling continuo
- **Acumulación de archivos:** hay que limpiar peticiones viejas
- **No es una arquitectura real:** nadie hace IPC por archivos en producción

Flujo de trabajo

Persona

¿Puede trabajar?

Todos

● Técnicamente sí, pero la experiencia de desarrollo es pésima

Tabla Comparativa

Opción	Dependencias nuevas	Performance	QA puede testear	Daniel sin WSL2	Complejidad	Madurez
0. Monolito	Ninguna	● Máxima	✗ Necesita Docker	✗ Bloqueado	● Mínima	✓ Actual
A. Qt Local Sockets	Ninguna ✓	● Alta	✓ Script simple	✓ Libre	● Baja	NEW Propuesta
B. TCP Local	Ninguna	● Alta	✓ curl/telnet	✓ Libre	● Baja	NEW Propuesta
C. HTTP REST	cpp-http lib + curl	● Media	✓ curl, Postman	✓ Libre	● Media	NEW Propuesta

Opción	Dependencias nuevas	Performance	QA puede testear	Daniel sin WSL2	Complejidad	Madurez
D. Shared Library	Ninguna	● Máxima	✗ Difícil	● Necesita .dll	● Media	 Propuesta
E. gRPC	gRPC+Protobuf	● Media	✓ grpcurl	● Build complejo	● Alta	 Propuesta
F. ZeroMQ	libzmq	● Alta	● Tools extra	✓ Libre	● Media	 Propuesta
G. Archivos	Ninguna	● Baja	✓ Trivial	✓ Libre	● Baja	 Propuesta

Leyenda de colores

Color	Significado
●	Punto fuerte de esta opción
●	Aceptable pero con salvedades
●	Debilidad significativa o bloqueante

Recomendación Final

🏆 Opción A — Qt Local Sockets (IPC Nativo)

¿Por qué?

1. **Cero dependencias nuevas.** Qt6::Core ya está en el proyecto. `QLocalServer` y `QLocalSocket` vienen incluidos.
2. **Daniel puede desarrollar frontend** en Windows nativo sin WSL2, sin Docker, sin complicaciones.
3. **QA puede testear el backend** escribiendo scripts simples que se conecten al named pipe.
4. **Es IPC nativo:** sin HTTP, sin web servers, sin overhead de red. Solo dos procesos hablando por un pipe del sistema operativo.
5. **Es la vía Qt:** signals/slots, event loop, `QJsonDocument` — todo es parte del framework que ya eligieron.
6. **Prepara para producción:** el backend como servicio y el frontend como app independiente es la arquitectura correcta para distribuir en Windows.

🥈 Opción B — TCP Local (alternativa ligera)

Si por alguna razón los named pipes no funcionan en el entorno de Daniel, TCP loopback es idéntico en concepto y esfuerzo, con la ventaja extra de que Manuel puede testear con `curl`.

🥉 Opción C — HTTP REST (si el equipo prefiere estándares web)

Si el equipo se siente más cómodo con REST, es una opción sólida. El costo es agregar `cpp-http-lib` como dependencia, pero gana en testabilidad y documentación.

Próximos pasos

Una vez elegida la opción:

1. **Propuesta formal** (documento de alcance, tareas, plan)
 2. **Diseño técnico** (interfaces, tipos, endpoints)
 3. **Reestructuración del CMake** (separar frontend y backend en targets independientes)
 4. **Implementación del middleware** (common, server, client)
 5. **Migración del Dockerfile** (backend service + Qt6::Core)
 6. **CI/CD** (build de backend + frontend para Windows)
 7. **QA protocol** (cómo Manuel testea cada componente)
-

Documento generado el 23 de junio de 2026. Próxima fase: Propuesta formal de implementación.